

Lineamientos de SCV distribuido con GIT y Azure Repos

Realizado Por	Fecha	Firma
Royss Hugo Martel Massi	24/03/2025	
Revisado Por	Fecha	Firma
Jonathan Franchesco Torres Baca	11/05/2025	
Aprobado Por	Fecha	Firma
Jose Jesus Salazar Silva	12/05/2025	

Versión

1.1.0

Objetivo

Establecer un marco de trabajo estandarizado para la gestión del código fuente utilizando Git con Azure Repos, garantizando eficiencia, calidad y colaboración entre equipos. Este documento proporciona lineamientos para el versionado de código, la organización de ramas y las mejores prácticas para un flujo de trabajo estructurado que contempla los desarrollos realizados dentro de Atlantic City.

Alcance

Estos lineamientos aplican a todas las áreas técnicas de la organización que gestionan código fuente mediante control de versiones. Asegurar un workflow eficiente, estandarizado y colaborativo en la gestión del versionamiento, independientemente del tamaño del proyecto o del framework metodológico adoptado.

Lineamiento de Repositorios

- Cada repositorio debe estar creado en el proyecto del equipo responsable.
- El nombre del repositorio debe estar en minúscula y separado por guiones.
- El nombre del repositorio debe cumplir con mínimo 5 y máximo 40 caracteres.
- Obligatorio incluir el archivo `README.md` que describa el propósito, estructura, dependencias y links de documentos de valor (arquitecturas, diagramas flujos, etc).
- Agregar un archivo `.gitignore` apropiado según la tecnología usada.
- La rama por defecto de cada repositorio será `main`.
- Utilizar nombres descriptivos y consistentes con la siguiente estructura `<nomenclatura>-<nombre-proyecto>-<nombre-adicional>`

Nomenclatura	Descripción
web	Para aplicaciones web (landings, admin, gestores, etc).
api	Para aplicaciones restfull, soap.
etl	Para aplicaciones de ETL.
whk	Para aplicaciones específicos que se comportaran como webhook.
ms	Para aplicaciones en microservicios.
mf	Para aplicaciones en microfrontend.
infra	Para aplicaciones netamente de infraestructura IaC.
pubsub	Para aplicaciones netamente que se comportaran como publicadores y/o subscriptores (colas, tópicos, streams, etc)
test-<tipo-testing>	Para aplicaciones de testing automatizados.
framework	Para aplicaciones que requieran ser integrados mediante servicios asociados.
model	Para aplicaciones enfocados al modelado de base de datos.
lib	Para aplicaciones que tienen como finalidad comportarse como una librería compartida.

✓ Ejemplos:

```

1 web-crmatlantic
2 api-jackpots
3 etl-ods-casino-online
4 whk-callbell
5 ms-reclamos-sala
6 mf-modulo-clientes
7 infra-securizacion-sanitizacion
8 api-securizacion-sanitizacion-svless
9 pubsub-send-email-customers
10 test-st-web-crmatlantic-mod-perfil
11 test-e2e-web-portalfinanzas
12 model-prd-mysql-data
13 framework-ingesta-datalake
14 lib-acity-components

```

Lineamientos de Ramas

- Las ramas deben estar vinculadas a tarjetas de trabajo en Azure Boards o contener el id de HU en su nombre.
- El nombre de la rama debe estar en minúscula y separado por guiones.
- Todas las fusiones a **main** y **dev** deben realizarse mediante **PRs** con revisión obligatoria.
- Prohibido el uso de **force push** en cualquier rama.

- Eliminar las ramas temporales luego de su fusión.
- La rama **main** debe tener un tag de versionamiento para cada pase a producción.
- No se debe realizar **commits** directos en la ramas principales.
- Se debe requerir al menos 1 aprobador para la rama **dev** 2 para **main**.
- Mantener actualizado siempre las rama dev y ramas temporales.
- Los **PRS** deben tener un titulo y una descripción detallada de las actividades que contemplan o en todo caso la HU asociada.
- Usar políticas de protección en ramas principales (**branch policies**). Debe contemplar mínimo las siguientes configuraciones:

```

1 - Branch Policies
2   Require a minimum number of reviewers
3   Check for comment resolution
4   Merge Type only squash merge
5 - Build Validation (1)
6 - Automatically included reviewers (1)

```

- Usar una convención estándar para los nombres de ramas: **<tipo>/[opcional-codigo-hu] - <descripcion>**.

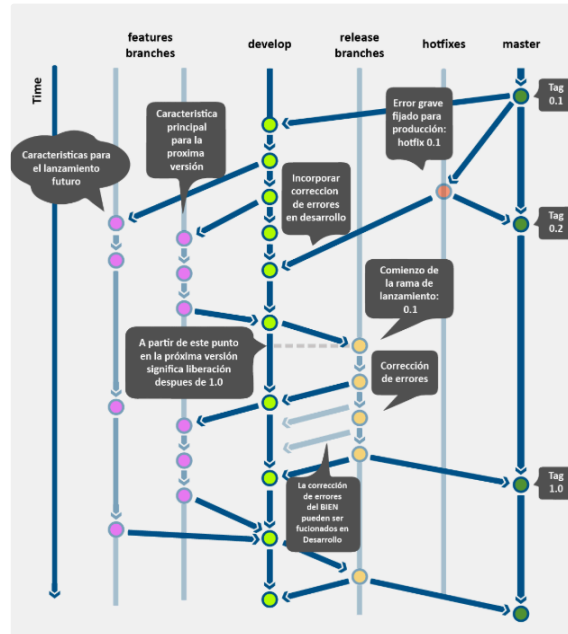
Ramas	Tipo	Descripción
main	principal	Código estable y desplegado en producción.
dev	principal	Rama base para integrar nuevas funcionalidades.
feature	temporal	Para desarrollar nuevas funcionalidades.
release	temporal	Para preparar una nueva versión (pre-producción).
hotfix	temporal	Para corregir errores urgentes en producción.
fix	temporal	Para corregir errores detectados en release o errores no críticos en producción.
spike	temporal	Para realizar ensayos (nuevas tecnologías, nuevas ideas).
chore	temporal	Para tareas de mantenimiento, no altera funcionalidades.
refactor	temporal	Para refactorizar código, reducir deuda técnica, mayor legibilidad, no altera funcionalidades.

```

1 feat/123-agregar-login
2 fix/456-correccion-error
3 hotfix/789-parche-produccion
4 chore/354-refactorizar-proceso-pago
5 release/1.0.0
6 spike/manejo-estado-zustand
7 refactor/proceso-facturacion

```

Flujo de trabajo



Ejemplo de flujo de trabajo con Gitflow:

1. Un desarrollador crea `feature/nueva-funcionalidad` desde la rama `dev`.
2. Tras finalizar la implementación, se abre un `Pull Request` y, tras pasar los filtros de aprobación, se fusiona en la rama `dev`.
3. Se crea una rama `release/1.2.0` cuando el código está listo para producción.
4. Tras pruebas exitosas, `release/1.2.0` se fusiona en `main` y se etiqueta `v1.2.0`.
5. Si se detecta un error en producción, se crea `hotfix/154-correccion-bug` desde `master` y se corrige.

Tags de Versionamiento

Se utiliza **Semantic Versioning (SemVer)** con el formato `Major.Minor.Patch`:

- **Major** (`X.0.0`): Cambios incompatibles con versiones anteriores.
- **Minor** (`X.Y.0`): Nuevas funcionalidades sin romper compatibilidad.
- **Patch** (`X.Y.Z`): Corrección de errores o mejoras menores.

✓ Ejemplos:

```

1 Corrección de errores (Patch)
2 1.0.0 → 1.0.1 → Se corrige un bug sin afectar la funcionalidad.
3 2.3.4 → 2.3.5 → Pequeña mejora o ajuste sin cambios de API.
4
5 Nueva funcionalidad retrocompatible (Minor)

```

```
6 1.0.0 → 1.1.0 → Se añade una nueva función sin romper compatibilidad.
7 2.3.4 → 2.4.0 → Se introduce un nuevo endpoint en la API.
8
9 Cambios incompatibles con versiones anteriores (Major)
10 1.0.0 → 2.0.0 → Se eliminan o modifican características de forma no ret
11 3.5.2 → 4.0.0 → Refactorización que rompe integraciones previas.
```

Lineamientos de Commits

- Mantener los `commits` limpios y con mensajes significativos.
- Usar el tiempo verbal imperativo en presente ("agrega", no "agregó" o "agregado").
- Revisar el código antes de hacer `commit`.
- Hacer `commits` pequeños y atómicos.
- Guiarse del uso de `conventional commits`.
- Usar `husky` como herramienta para estandarizar `commits`.
- Se recomienda usar un archivo `CHANGELOG.md` con formato tipo Keep a Changelog.
- Se debe usar la estructura de `conventional commits`.

```
1 category(scope or module): message
2 [optional body]
3 [optional footer(s)]
```

Convenciones de Commits

Antes de crear una solicitud de incorporación de cambios, verifique si sus confirmaciones cumplen con las convenciones de este repositorio.

Al crear una confirmación, le rogamos que siga la convención `category(scope or module): message` en su mensaje de confirmación, utilizando una de las siguientes categorías:

Categoría	Propósito
<code>feat / feature</code>	todos los cambios que introducen código completamente nuevo o nuevas características.
<code>fix</code>	cambios que corrigen un error (idealmente, también se debe hacer referencia a un problema si existe).
<code>refactor</code>	cualquier cambio relacionado con el código que no sea una corrección ni una característica.
<code>docs</code>	modificar la documentación existente o crear nueva (ej., README, documentación para el uso de una biblioteca o CLI).
<code>build</code>	todos los cambios relacionados con la compilación del software, cambios en las dependencias o la adición de nuevas dependencias.
<code>test</code>	todos los cambios relacionados con las pruebas (añadir nuevas pruebas o modificar las existentes).

ci	todos los cambios relacionados con la configuración de la integración continua (p. ej., acciones de GitHub, sistema CI).
chore	todos los cambios en el repositorio que no encajan en ninguna de las categorías anteriores.

✓ Ejemplos:

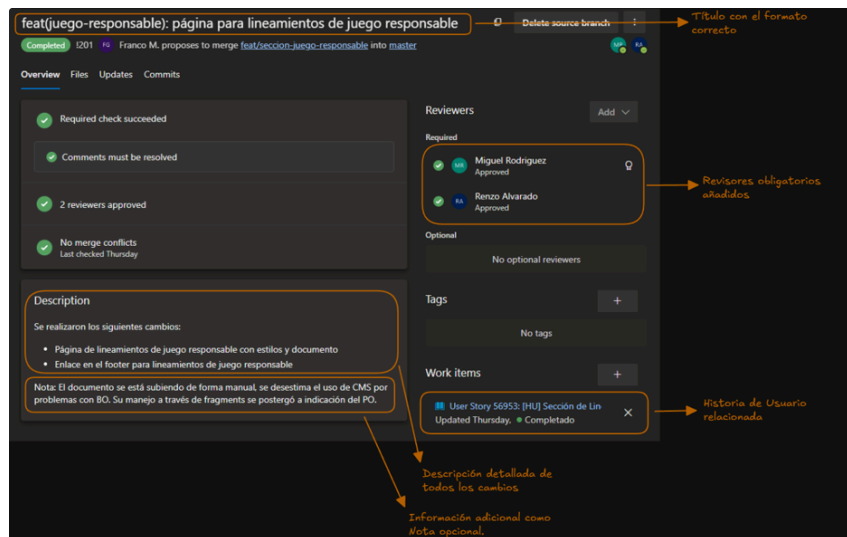
```

1 feat: Se cambia logica de puntos para el sorteo de tus sueños
2 Se cambia la fuente de origen de los puntos, ahora se aplica la partici
3 de acuerdo a los niveles detallados en el brief 03-2025.
4 Tarea: 1542
5
6 chore: Se mejora la configuracion de variables de entorno con la librer
7 -Se agrega archivo config/envs.ts para validar y configurar variables d
8 Tarea: 5468
9
10 build: Se agrega libreria de dotenv y joi para controlar y validar vari
11 de entorno
12
13 hotfix: Se corrige el header para dispositivos móviles.
14 Tarea: 3547
15
16 bugfix: Se agrega validación cuando el producto es null, se envia mensa
17 excepción.
18 Tarea: 4567

```

Lineamiento de Pull Request (PR)

- Debe tener un titulo descriptivo y breve con el siguiente formato `<tipo-rama>: <descripcion>`.
- El titulo debe estar en minúscula y separado por guiones.
- El contenido debe contemplar una descripción de las tareas que se van a desarrollar.
- El contenido debe contemplar el nombre y correo del PM o PO responsable.
- Se recomienda asignar mínimo 1 revisor para la rama dev y 2 para la rama `main`.
- Se debe contemplar un `build success` como política de aprobación.
- Se debe resolver todas las observaciones realizados por los revisores como política de aprobación.
- Se debe asignar una HU o caso contrario indicarlo dentro de la descripción `[id-hu]-[proyecto-origen]`.
- Opcionalmente se debe agregar tags relacionados al PR.



Conexiones Remotas y Autenticación

Configuración de Conexiones Remotas

Protocolo	
HTTPS	Todas las conexiones remotas a Azure DevOps deben realizarse mediante protocolo HTTPS
SSH	Solo para usuarios con autorización específica y configuración de claves aprobadas por el equipo de seguridad.

Lineamientos de Conexiones

- Responsabilidad individual por cada desarrollador, se debe generar su propio PAT desde su cuenta personal de Azure DevOps.
- El administrador NO debe generar PATs para otros usuarios, Esto garantiza:
 - Trazabilidad individual de las acciones.
 - Responsabilidad personal sobre la seguridad del token.
 - Cumplimiento de auditoría y controles de acceso.
- El tiempo de expiración de cada PAT (Personal Access Token) debe ser mínimo 30 días y máximo 90 días, ningún PAT debe ser personalizado con tiempos largos.
- Renovar las contraseñas de los PAT (Personal Access Token) según su tiempo previsto.
- Los PAT deben almacenarse en un Gestor de Credenciales.
- Otorgar únicamente los permisos necesarios para la integración.

Buenas Prácticas

- Usar nombres descriptivos y estandarizados en las ramas.
- Eliminar las ramas temporales una vez fusionado mediante los PRs para mantener un repositorio limpio.
- Mantener **commits** pequeños y bien documentados.
- Aplicar **squash** en lugar de **merge** para limpiar el historial cuando sea necesario.

- Etiquetar versiones en `main` para un mejor rastreo.
- Integrar herramientas de calidad y seguridad en el flujo de trabajo. (husky, eslint, prettier, commitlint).
- Configurar `branch policies` para evitar fusiones accidentales sin revisión.

Conclusión

El uso de Git con Azure Repos bajo estos lineamientos garantiza un flujo de trabajo eficiente, colaborativo y estructurado. La correcta organización de ramas, el versionamiento estándar y las buenas prácticas permiten mejorar la trazabilidad, calidad y seguridad del código. La adopción de herramientas como Azure DevOps y GitFlow refuerza la gestión del ciclo de vida del software, minimizando errores y facilitando la integración continua. Se recomienda revisar periódicamente este lineamiento para adaptarlo a las necesidades de la organización y optimizar los procesos de desarrollo.